Leo Marcinkus

VLSI - ECEN 4012

Project 1 Report

03/14/26

Introduction:

The circuit we're meant to create uses a VGA (Variable Gain Amplifier) with and AGC (Automatic Gain Control) combined with a "mixed-signal" architecture. I have attempted the "More Verilog, less analog" version. The circuit takes in an analog sine wave input with its amplitude controlled by a defined volume signal.

Instead of doing the gain adjustment entirely in an analog implementation, the circuit digitizes the relevant signals and performs the scaling digitally in a VerilogA block before converting the output back to analog (I tried a whole SystemVerilog/Verilog block but it did not go well with the Spectre simulator).

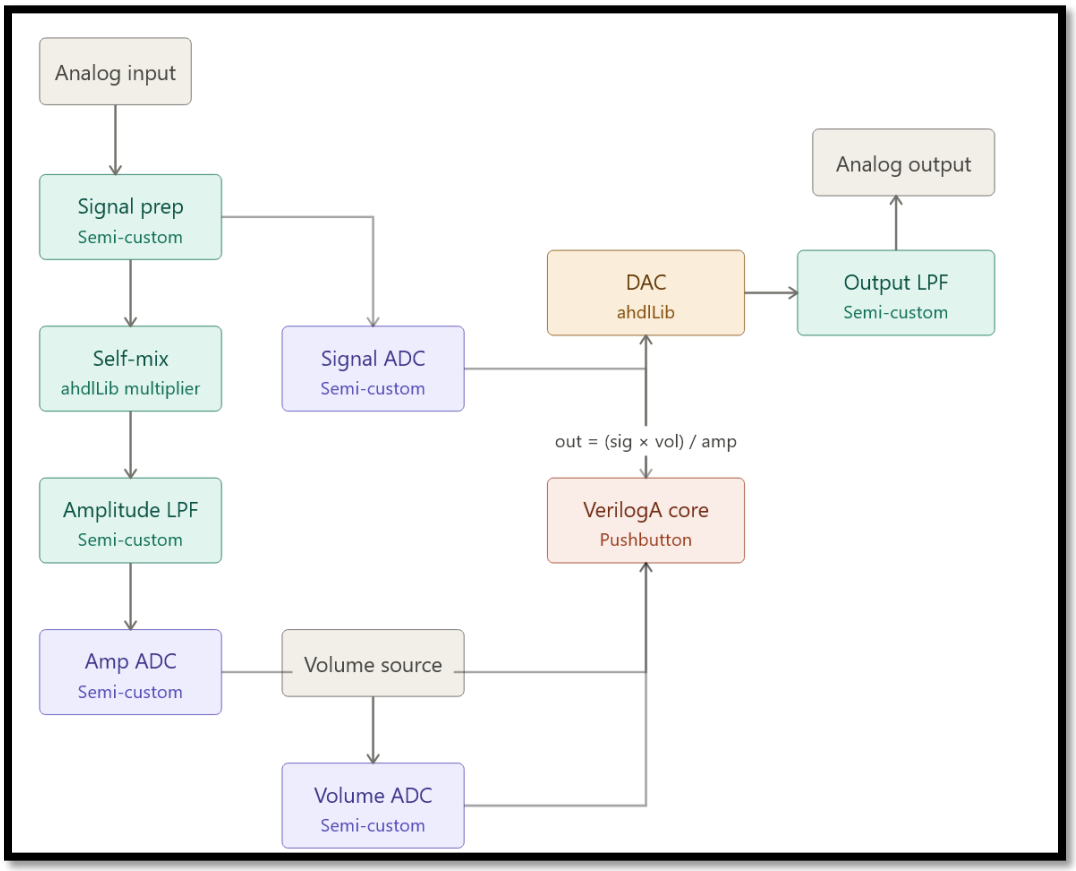


Figure 1: Overall System Block Diagram

Block Descriptions:

Analog Input Sine - The input stimulus. A sinusoidal voltage source representing a real-world signal with varying amplitude.

Input Signal Preparation - Semi-Custom Flow — AC-couples the input through a 100nF capacitor and biases it to the mid-supply point of 0.9V using a 100kΩ/100kΩ voltage divider. This ensures the signal stays within the 0–1.8V operating range of the sky130 process regardless of input offset.

Multiplier- ahdlLib - Multiplies the prepared signal by itself using an ideal multiplier with gain of 8. Using the trigonometric identity $\sin(A)\times\sin(A) = A^2/2 \times (1 - \cos(2ft))$, squaring the signal produces a DC component proportional to the square of the input amplitude plus a double-frequency AC component.

Amplitude Low-Pass Filter - Semi-Custom Flow - A 100kΩ resistor and 1µF capacitor form a low-pass filter with a 100ms time constant. This filters out the double-frequency AC component from the squarer output, leaving only the DC component proportional to input amplitude. This DC value feeds the amplitude ADC.

ADC for Input Signal - Semi-Custom Flow - A 3-bit flash ADC that converts the prepared sine wave into a 3-bit digital code (sig0, sig1, sig2). It consists of a resistor ladder, seven comparators, and an XOR encoder.

ADC for Amplitude - Semi-Custom Flow - An identical 3-bit flash ADC that converts the amplitude estimate DC voltage into a 3-bit digital code (amp0, amp1, amp2).

Analog Volume Control - A DC voltage source representing the user's volume setting, analogous to a volume knob. The voltage ranges from 0V to 1.8V.

Volume-to-Digital ADC - Semi-Custom Flow - An identical 3-bit flash ADC that converts the volume control voltage into a 3-bit digital code (vol0, vol1, vol2).

VerilogA VGA/AGC Core - Pushbutton Flow - The digital gain control block. Takes the three 3-bit inputs and computes the scaled output using the formula: out = (sig × vol) / amp. If amp equals zero the output is forced to zero to prevent division by zero. The result is saturated to a maximum of 7 to prevent overflow. Implemented in VerilogA for compatibility with Spectre AMS simulation.

DAC - ahdlLib - An ideal 8-bit DAC from the ahdlLib library. The 3-bit output from the VerilogA block connects to the most significant bits (vd5, vd6, vd7) to maximize output voltage range. vref is set to 1.8V. Converts the digital scaled value back to an analog voltage.

Output Low-Pass Filter - Semi-Custom Flow - A 1kΩ resistor and 1µF capacitor smooth the stepped DAC output into a more continuous analog waveform. The 1ms time constant is appropriate for a 1kHz input signal.

Analog Output - The final scaled analog sine wave whose amplitude is determined by the volume control input.

Block 1 - Input, Multiplier, and LPF:

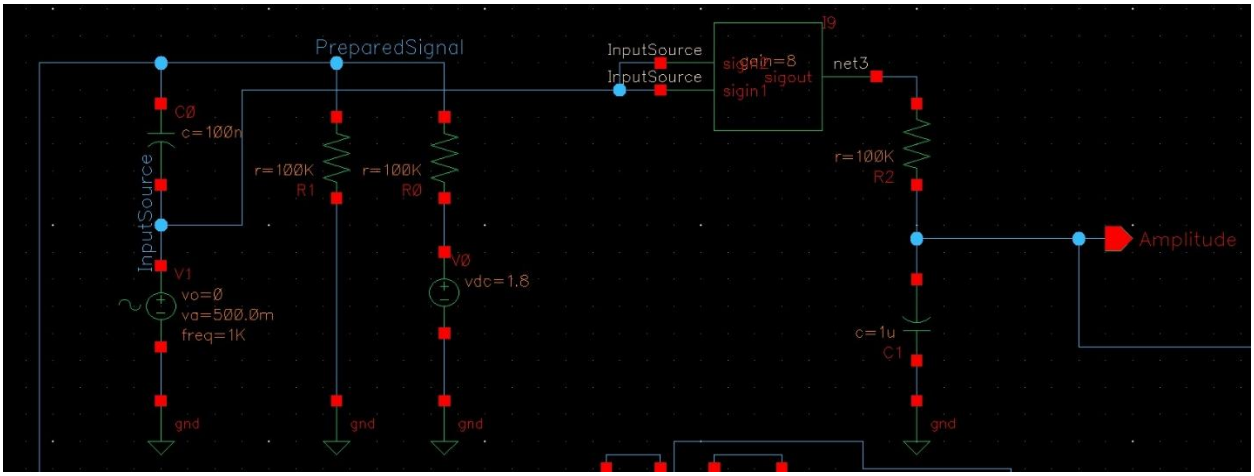


Figure 2: Block 1 Schematic

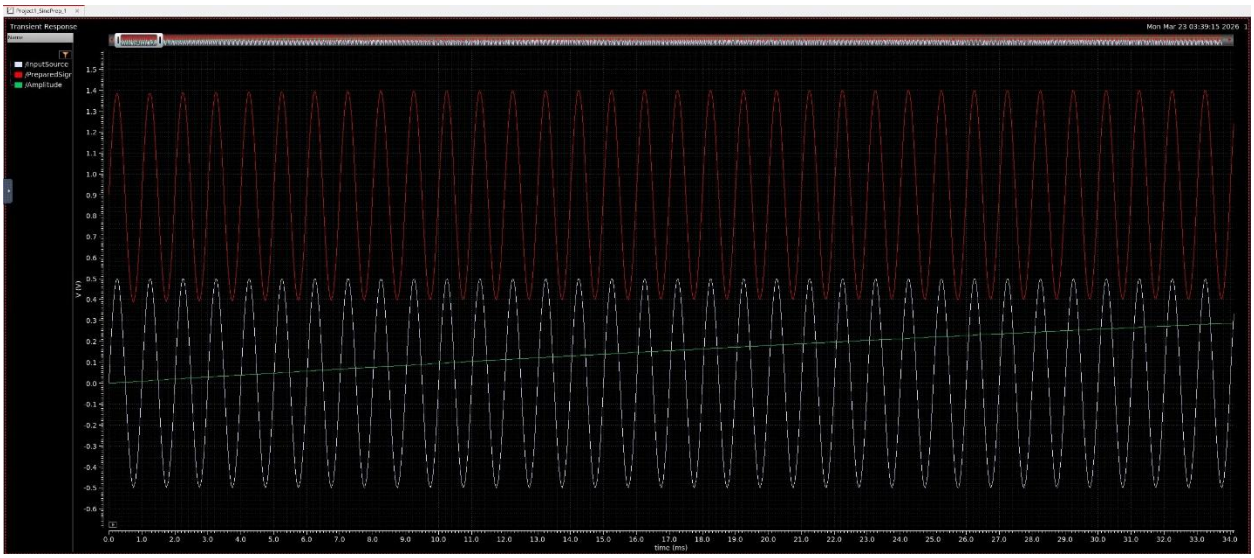


Figure 3: Block 1 Simulation (Zoomed In)

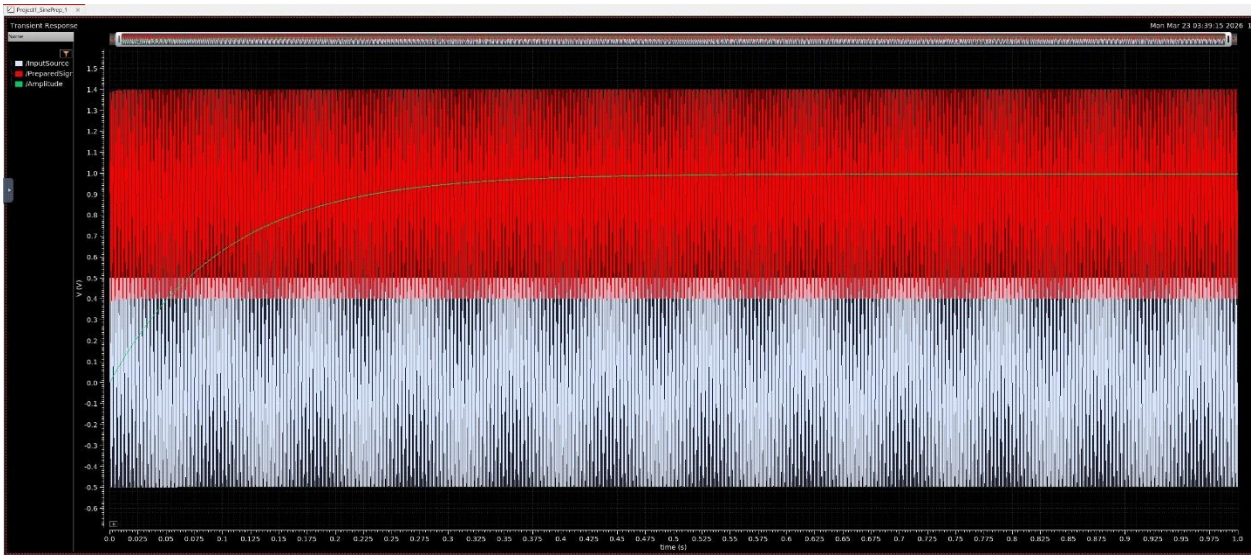


Figure 4: Block 1 Simulation (Zoomed Out)

Block 1 Simulation Results:

Schematic:

100 nF AC coupling capacitor (C0)

100 kΩ / 100 kΩ voltage divider (R0, R1) biasing to 0.9V mid-supply

1.8V DC source

Multiplier (gain = 8) squaring the prepared signal

100 kΩ / 1 μF low-pass filter producing the Amplitude estimate

Figure 3:

InputSource - White, ±500mV centered at 0V, frequency set to 1kHz, raw input

PreparedSignal - Red, biased up to swing between ~0.4V and ~1.4V (Shifts the input into the 0-1.8V operating range)

Amplitude - Green, slowly charging up from 0V toward its steady state, the RC filter is charging

Figure 4:

The Amplitude (Green) settles to approximately 1.0V after ~300ms. This is the DC estimate of input amplitude from the self-mix path

PreparedSignal (Red) oscillates at 1kHz

Block 2 - ADCs:

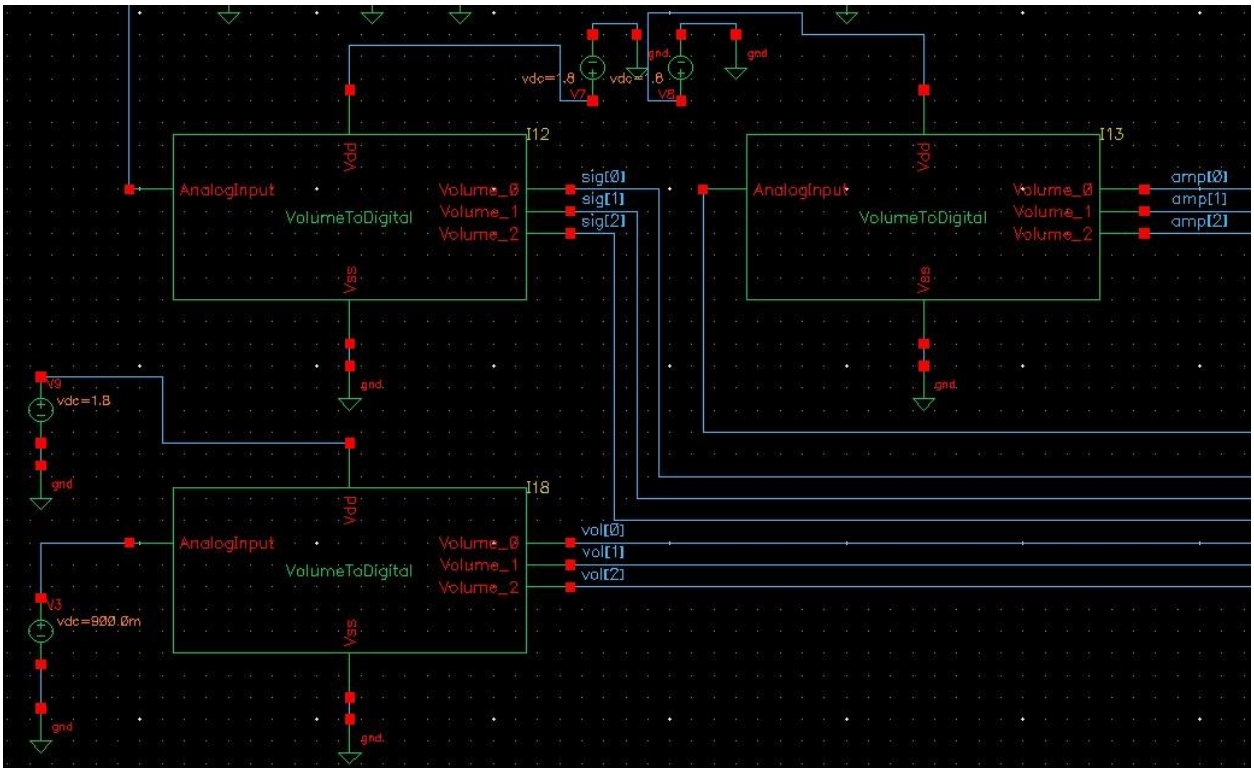


Figure 5: Block 2 Schematic

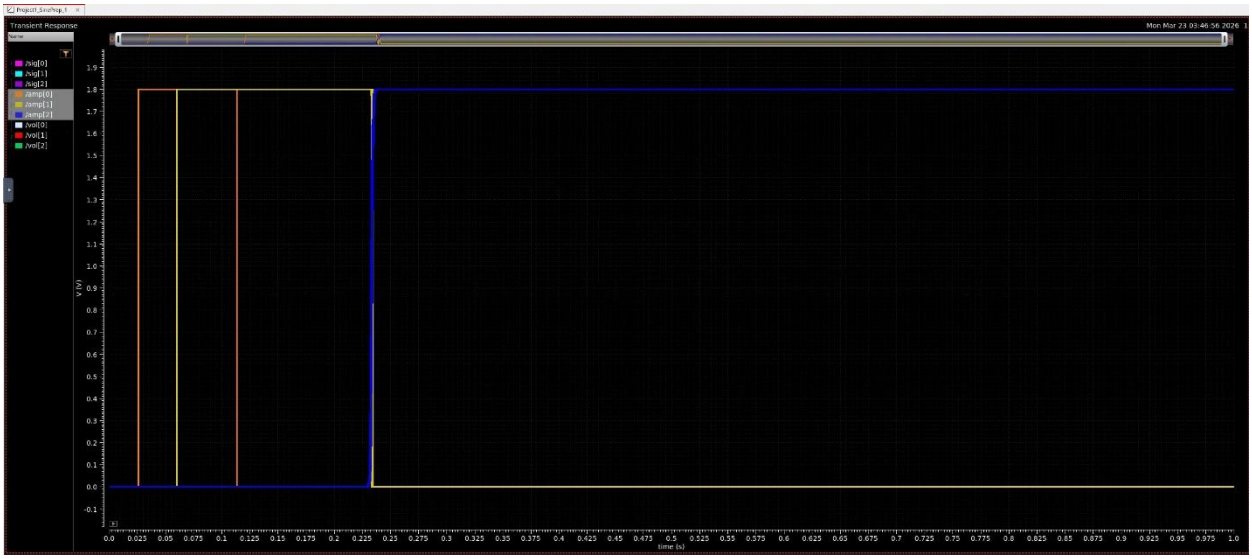


Figure 6: Block 2 Amp Simulation

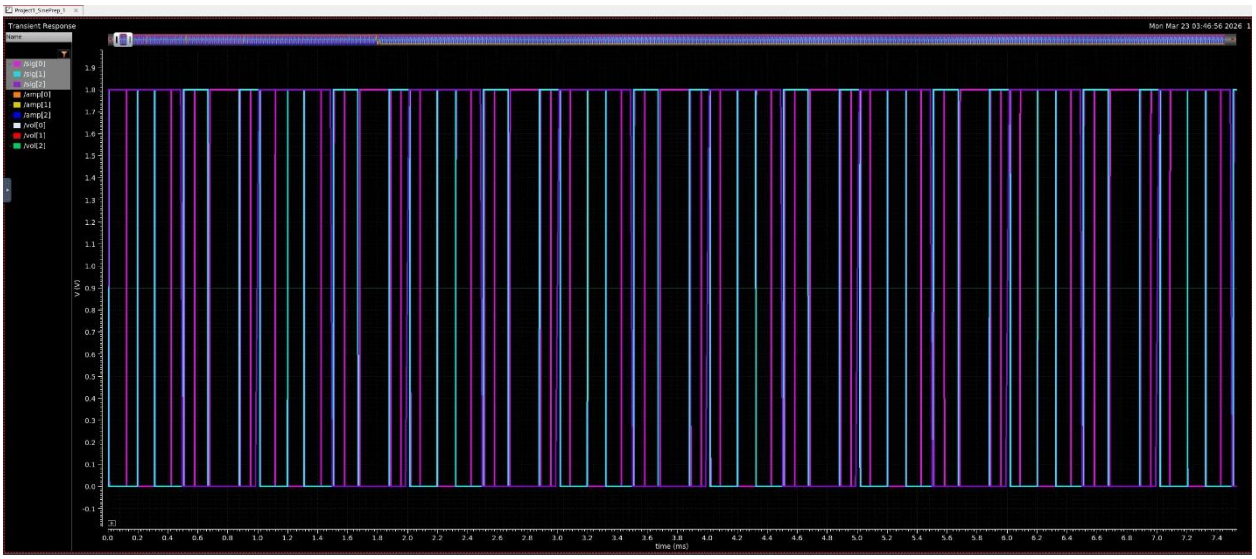


Figure 7: Block 2 Sig Simulation

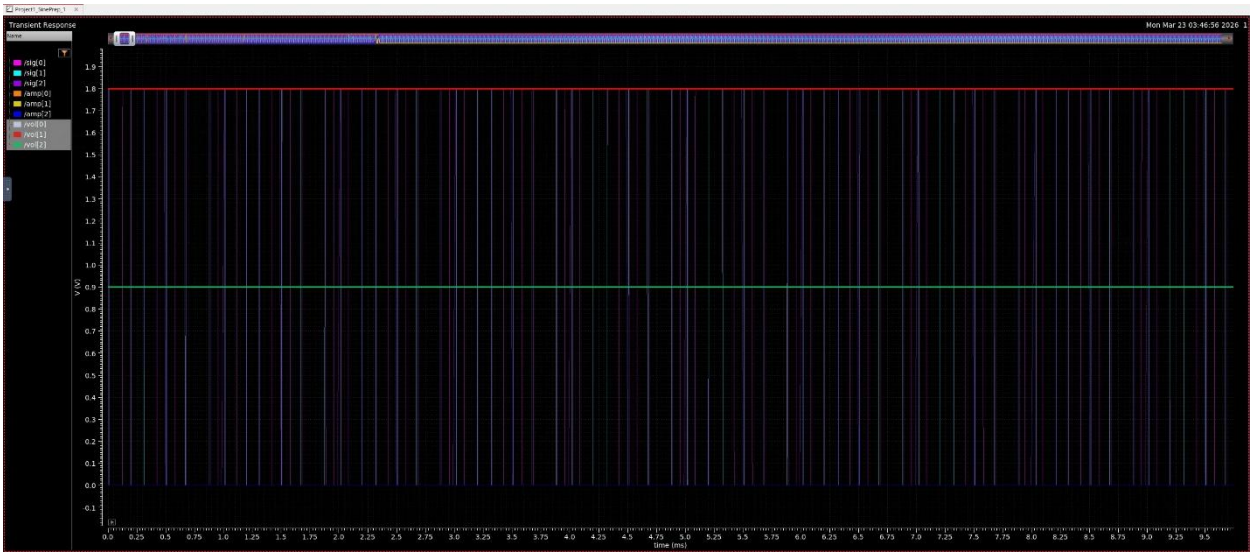


Figure 8: Block 2 Vol Simulation

Block 2 Simulation Results:

Schematic:
Three instances of the VolumeToDigital flash ADC block, each with its own independent 1.8V supply. I12 receives PreparedSignal (sine), I13 receives the Amplitude node, and I18 receives the 0.9V DC volume knob. Each outputs a 3-bit digital code.

Figure 6 Amp ADC:

Amp0 - (Orange) and amp1 (blue) transition from 1.8V to 0V at around 230ms

Amp2 - (Yellow) stays low throughout

The amplitude RC filter charges up past comparator thresholds (in my ADC schematic/symbol), the binary encoding shifts. The amp bits changing over time reflects the amplitude estimate settling, which is expected behavior.

Figure 7 Sig ADC:

sig0, sig1, sig2 - (Pink, cyan, purple) toggle rapidly between 0V and 1.8V

The flash ADC is continuously sampling the 1kHz sine wave, so the output bits switch every time the sine crosses a comparator threshold. The pattern of switching represents the instantaneous digital encoding of the sine wave amplitude at each moment.

Figure 8 Vol ADC:

vol0 - (White) stays steady at 1.8V

vol1 - (Red) stays steady at 1.8V

vol2 - (Green) sits at 0.9V. The overall circuit still functions even with this one bit stuck at 0.9V. I couldn't find the root cause. The analog input to the vol ADC is a set 0.9V DC input. So the two bits firmly at 1.8V still confirm the ADC is working and encoding the volume input. If I change that set analog input to a lower or higher value, the outputs change so the block is still functioning despite one bit remaining off.

Block 3 - Verilog, DAC, Final output

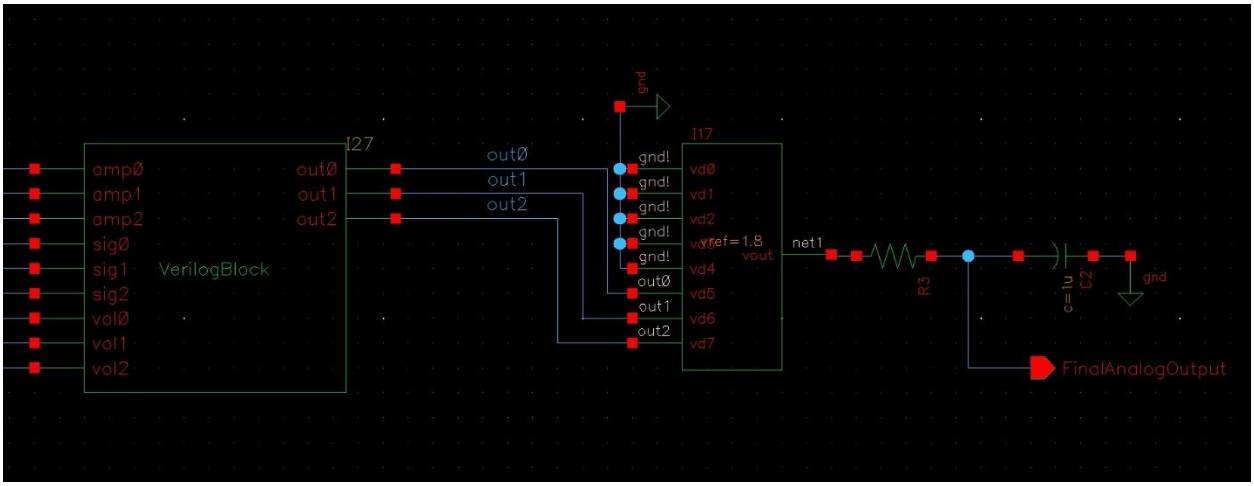


Figure 9: Block 3 Schematic

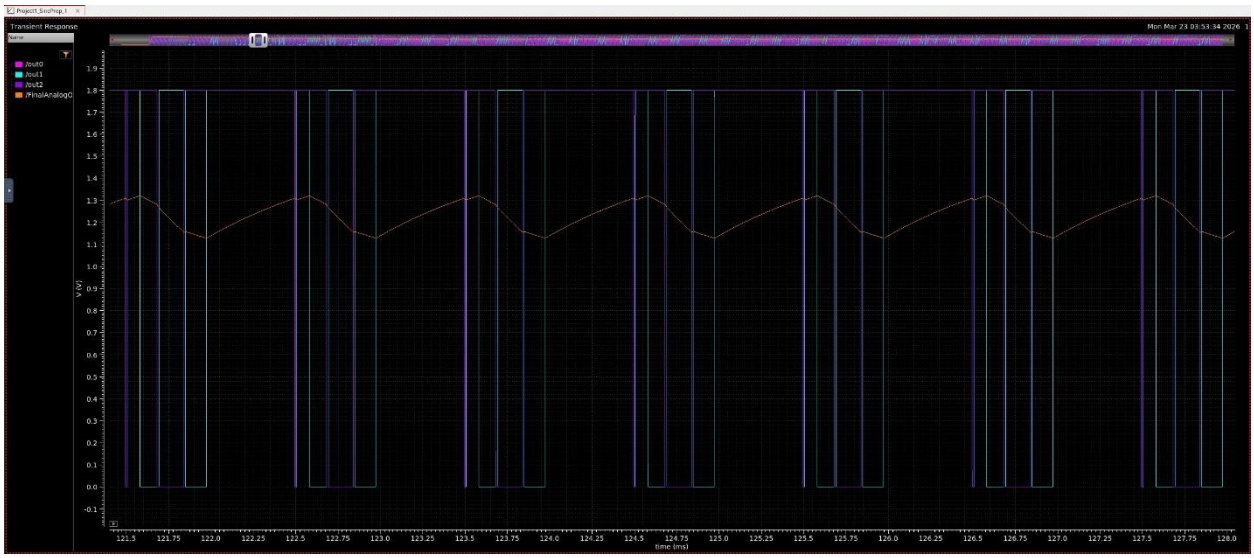


Figure 10: Block 3 Schematic (Zoomed In)

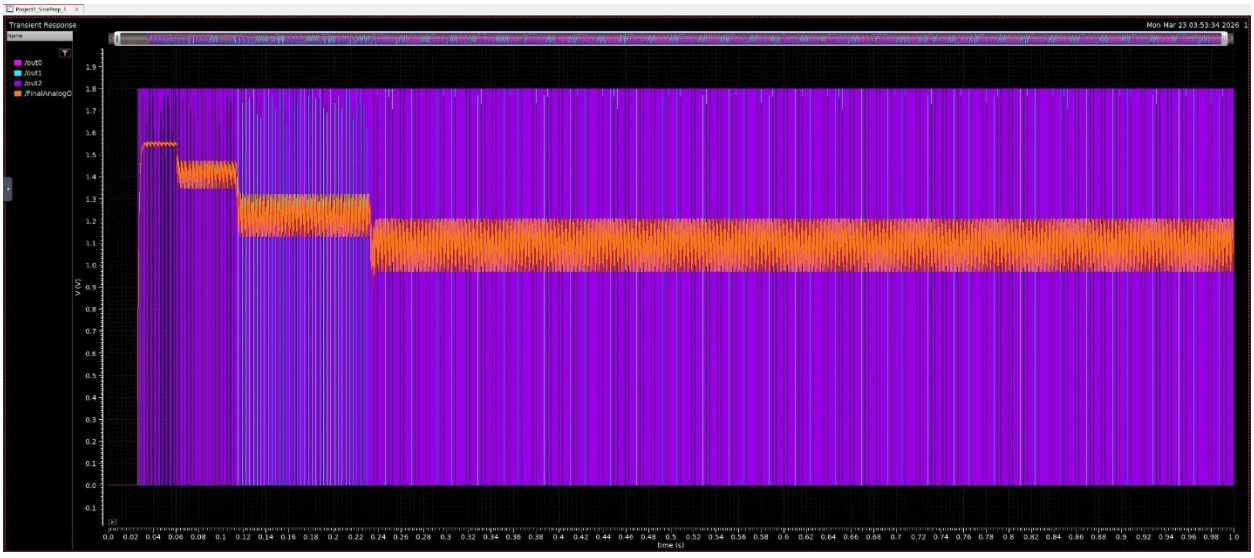


Figure 11: Block 3 Schematic (Zoomed Out)

Block 3 Simulation Results:

Schematic - The VerilogA block (I27) receives nine single-bit inputs, amp0/1/2, sig0/1/2, and vol0/1/2. It produces three output bits, out0/1/2. These connect to the MSBs of the dac_8bit_ideal from the ahdlLib library (vd5, vd6, vd7) with vd0-vd4 tied to ground. The DAC output feeds through a 1kΩ resistor and 1µF capacitor low-pass filter to FinalAnalogOutput.

Verilog Code (Can be found after this section) - The core logic implements the formula out = (sig × vol) / amp. The three integer inputs are reconstructed from their individual bits using weighted sums.

You'll see: sig_int = (V(sig2)>0.5 ? 4:0) + (V(sig1)>0.5 ? 2:0) + (V(sig0)>0.5 ? 1:0).

A divide-by-zero guard forces output to zero when amp_int = 0. The result is saturated to 7 to prevent overflow, then each output bit is driven to either 1.8V or 0V. Originally, I used SystemVerilog, converted to Verilog, using the digital Cadence tools like Genus. I created a symbol in Virtuoso, but the Spectre simulator could not handle a mixed system. I had to convert it to VerilogA for compatibility since the currently installed Xcelium version was incompatible with Virtuoso's import flow.

Figure 10:

Zoomed in at ~122–128ms

FinalAnalogOutput - (Orange) shows a clear sinusoidal waveform oscillating between ~1.1V and 1.3V. The DAC stepping is visible as small discontinuities, smoothed by the output LPF. The outputs out0/1/2 are toggling correctly in response to the scaled signal which confirms the circuit is working. The sine wave shape is reconstructed at the output but at a very low resolution.

Figure 11:

Over 1 second

FinalAnalogOutput - (Orange) shows four distinct amplitude levels stepping down from ~1.55V to ~1.5V to ~1.3V to ~1.2V before settling. The amplitude RC filter charges and amp_int increases through values 1 to 2 to 3 to 4. The denominator of (sig × vol) / amp increases which reduces the output gain. The circuit is correctly adjusting the output amplitude when given the measured input amplitude. After ~250ms, the output reaches a stable oscillating band around 1.0-1.2V and settles.

Verilog code:

```
`include "constants.vams"
`include "disciplines.vams"

module VolumeControl (sig0, sig1, sig2, amp0, amp1, amp2, vol0, vol1, vol2,
out0, out1, out2);

    input sig0, sig1, sig2;
    input amp0, amp1, amp2;
    input vol0, vol1, vol2;
    output out0, out1, out2;

    electrical sig0, sig1, sig2;
    electrical amp0, amp1, amp2;
    electrical vol0, vol1, vol2;
    electrical out0, out1, out2;

    integer sig_int, amp_int, vol_int, scaled;

    analog begin
        sig_int = (V(sig2) > 0.5 ? 4 : 0) + (V(sig1) > 0.5 ? 2 : 0) +
(V(sig0) > 0.5 ? 1 : 0);
        amp_int = (V(amp2) > 0.5 ? 4 : 0) + (V(amp1) > 0.5 ? 2 : 0) +
(V(amp0) > 0.5 ? 1 : 0);
        vol_int = (V(vol2) > 0.5 ? 4 : 0) + (V(vol1) > 0.5 ? 2 : 0) +
(V(vol0) > 0.5 ? 1 : 0);

        if (amp_int == 0)
            scaled = 0;
        else begin
            scaled = (sig_int * vol_int) / amp_int;
            if (scaled > 7) scaled = 7;
        end

        V(out0) <+ (scaled & 1) ? 1.8 : 0.0;
        V(out1) <+ (scaled & 2) ? 1.8 : 0.0;
        V(out2) <+ (scaled & 4) ? 1.8 : 0.0;
    end
endmodule
```

Overall Test/Full Circuit:

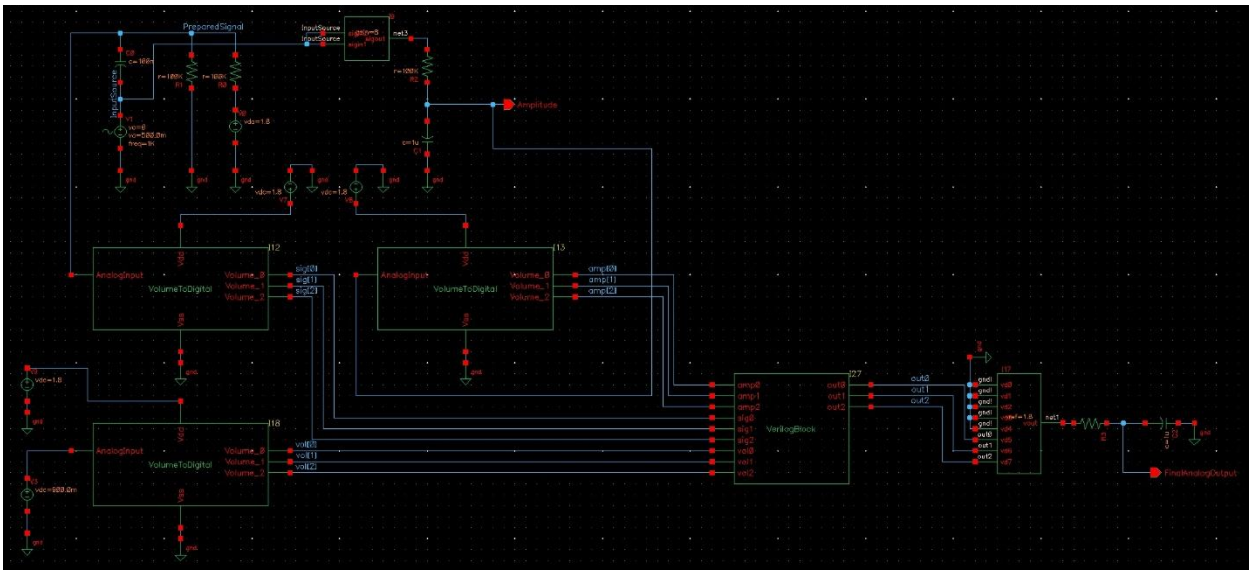


Figure 12: Full Circuit Schematic

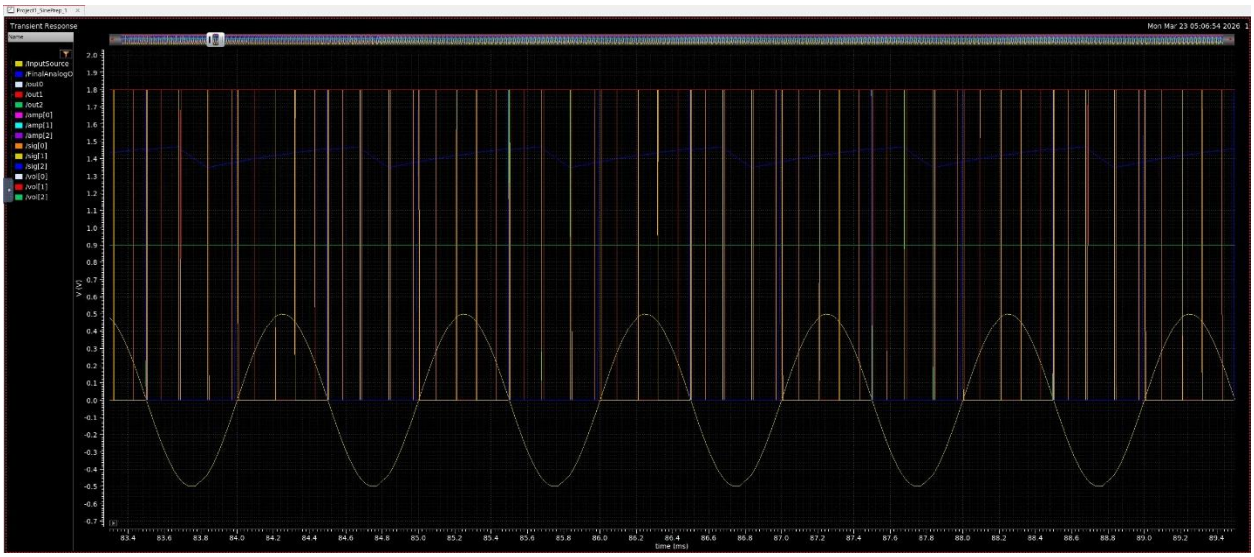


Figure 13: Full Circuit Simulation

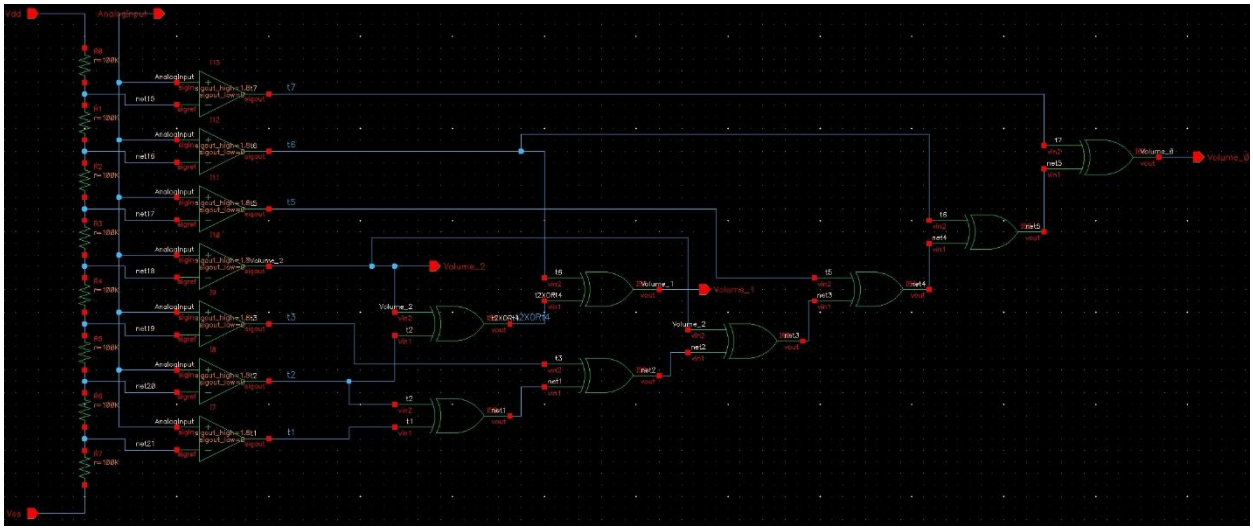


Figure 14: Full ADC Schematic (Volume to Digital Symbol)

Final Comments:

Did your circuit succeed or fail? Why?

Overall, my circuit functions correctly but only partially succeeded. The whole process works as intended. An analog sine wave is digitized, manipulated, and converted back into an analog signal. However, there are a few issues. The output does not show a perfectly reconstructed sine wave. It's more sharp and jagged than a true curve. I attribute this to my low resolution 3-bit ADCs I constructed. My design also does not include a clock, so the circuit is constantly sampling instead of at timed intervals which contributes to some noisy behavior when leaving the DAC.

What would you do differently if you were going to do it again?

Had I realized there was an included ideal 8-bit ADC in the ahdlLib library, I would have used that to start. Instead, I ended up creating my own 3-bit ADC without a clock. I thought it would make the process simpler, but it only produced less satisfactory results and more headaches.

What was valuable to you in this project?

The documentation for Virtuoso and the components in the various libraries used were by far the most helpful. I ran into many issues when trying to simulate my various blocks as I continued creating it. Fortunately, most of Virtuoso is well documented and I was able to get everything functioning with enough troubleshooting. The previous labs, 4 and 5, were also valuable for when I needed to be walked through the first steps again when designing a circuit in Virtuoso.

Data:

Final Error:

When the volume input was increased from 0.9V to 1.35V (a 1.5x increase), the steady-state output amplitude showed little measurable change. It remained at around ±0.125V peak-to-peak. I believe the cause of this is the 3-bit ADC resolution limitation. With only 8 discrete levels, small changes in volume input may hit the ADC thresholds. This will lead to little change in vol_int in Verilog and therefore little change in output scaling.